

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Conexión

Tema y Dataset

Ejercicios Prácticos Integrados

Cierre y Retroalimentación

Conclusiones

Introducción y Conexión

- **Semana 8, Sesión 1 (P1):**
 - Fundamentos de **pandas**: Series, DataFrames, selección, filtrado, groupby.
 - Limpieza de datos (NaN), estadísticas y visualización con Matplotlib.
- **Objetivo de hoy:** Aplicar estos conceptos en un caso integrado de **exoplanetas** con datos sintéticos, faltantes, derivaciones físicas y gráficas.

- **Consolidar** lectura, limpieza y análisis con pandas.
- **Aplicar** derivaciones físicas simples para exoplanetas.
- **Desarrollar** criterio para tratar datos faltantes (MCAR/MNAR).
- **Integrar** visualización y exportación de resultados.

Tema y Dataset

Contexto

- Modelaremos un **catálogo sintético** de exoplanetas alrededor de estrellas tipo solar.
- Variables clave:
 - **star_teff** [K], **star_radius_Rsun** [$R_{\odot}$], **star_metallicity** [dex]
 - **period_days** [d], **planet_radius_Re** [$R_{\oplus}$]
 - **a_AU** [UA] (ley de Kepler), **depth_ppm** [ppm] (profundidad de tránsito)
 - **snr** (señal-ruido observacional aprox.), **teq_K** [K] (temperatura de equilibrio)
- Inyectaremos **datos faltantes** MCAR/MNAR para practicar limpieza.

Meta: Progresión de inspección $\rightarrow$ limpieza $\rightarrow$ derivaciones $\rightarrow$ filtrado físico $\rightarrow$ visualización $\rightarrow$ exportación.

Tema y Dataset $\Rightarrow$ Generación del dataset sintético:

Exoplanetas por tránsito



```
1 import numpy as np
2 import pandas as pd
3
4 rng = np.random.default_rng(42)
5 n = 1200
6
7 # Estrellas
8 star_teff = np.clip(
9     rng.normal(5700, 800, n), 3500, 7500
10 ) # K
11 star_radius_Rsun = np.clip(
12     rng.lognormal(mean=np.log(1.0), sigma=0.25,
13         ↪ size=n), 0.6, 2.5
14 ) # R_sun
15 star_metallicity = rng.normal(0.0, 0.2, n) #
16     ↪ [Fe/H] dex
17 star_mass_Msun = np.clip(
18     rng.lognormal(np.log(1.0), 0.20, n), 0.5, 1.6
19 ) # M_sun
20
21 # Planetas
22 # Periodo ~ lognormal (mediana ~20 d)
23 period_days = rng.lognormal(
24     mean=np.log(20), sigma=0.9, size=n
```

```
24 # Ley de Kepler en unidades astronómicas (aprox):
25 # a[AU] = (P/365.25)^(2/3) * M_star^(1/3)
26 a_AU = (period_days/365.25)**(2/3) *
27     ↪ (star_mass_Msun**(1/3))
28
29 # Profundidad de tránsito (ppm) ~ (Rp/R*)^2
30 Rsun_to_Re = 109.1
31 depth_ppm = ((planet_radius_Re / (star_radius_Rsun
32     ↪ * Rsun_to_Re))**2) * 1e6
33
34 # SNR aprox: depth/sigma * sqrt(Ntr), sigma ~
35     ↪ lognormal
36 sigma_ppm = rng.lognormal(mean=np.log(300),
37     ↪ sigma=0.4, size=n)
38 Ntr = np.maximum(1, (3*365.25)/period_days) # 3
39     ↪ años misión
40 snr = depth_ppm / sigma_ppm * np.sqrt(Ntr)
41
42 # Dataset base
43 df = pd.DataFrame({
44     "id": [f"EP{idx:04d}" for idx in range(1,
45     ↪ n+1)],
46     "star_teff": star_teff,
47     "star_radius_Rsun": star_radius_Rsun,
48     "star_metallicity": star_metallicity,
49     "star_mass_Msun": star_mass_Msun,
50     "period_days": period_days,
51     "planet_radius_Re": planet_radius_Re,
52     "b": b,
53     "a_AU": a_AU,
54     "depth_ppm": depth_ppm,
```


Ejercicios Prácticos Integrados



Enunciado

- Muestra `df.shape`, `df.info()` y `df.describe()`.
- Lista columnas numéricas y no numéricas.
- Verifica unidades físicas de variables clave.

Conceptos: Inspección inicial y verificación de consistencia.

Ejercicios Prácticos Integrados $\ni$ Solución 1 de Referencia: Inspección y unidades



```
1 print("shape:", df.shape)
2 print("="*40); df.info()
3 print("="*40); print(df.describe())
4
5 num_cols = df.select_dtypes(include='number').columns.tolist()
6 non_num_cols = df.select_dtypes(exclude='number').columns.tolist()
7 print("Numéricas:", num_cols)
8 print("No numéricas:", non_num_cols)
9
10 # Unidades:
11 # star_teff[K], star_radius_Rsun[R_sun], star_mass_Msun[M_sun],
12 # period_days[d], planet_radius_Re[R_earth], a_AU[AU], depth_ppm[ppm]
```

Discusión: Asegurar unidades y tipos guía decisiones de limpieza y derivaciones.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 2:

Limpieza y datos faltantes



Enunciado

- Cuenta nulos por columna y porcentaje.
- Imputa `star_teff` con la mediana por cuartiles de `star_radius_Rsun`.
- Imputa `planet_radius_Re` usando profundidad:
$$\hat{R}_p \approx \sqrt{\text{depth_ppm}/10^6} \cdot R_{\star} \cdot 109.1.$$
- Imputa `star_metallicity` con la mediana global.

Objetivo: Practicar estrategias basadas en física y en agrupaciones.

Ejercicios Prácticos Integrados $\ni$ Solución 2 de Referencia:

Limpieza y datos faltantes



```
1 # Resumen de nulos
2 nulls =
  ↳ df.isnull().sum().sort_values(ascending=False)
3 pct = df.isnull().mean().round(3)
4 print(nulls); print(pct)
5
6 # Imputación star_teff por cuartiles de radio
  ↳ estelar
7 bins = pd.qcut(df['star_radius_Rsun'], 4,
  ↳ duplicates='drop')
8 teff_med = df.groupby(bins)['star_teff'].transform(
9     lambda s: s.fillna(s.median())
10 )
11 df['star_teff'] = df['star_teff'].fillna(teff_med)
12
13 # Imputación planet_radius_Re desde profundidad
14 # Rp[Re] = sqrt(depth/1e6) * Rstar[Rsun] * 109.1
15 need_rp = df['planet_radius_Re'].isna() &
  ↳ df['depth_ppm'].notna() &
  ↳ df['star_radius_Rsun'].notna()
16 Rp_est = np.sqrt(df.loc[need_rp, 'depth_ppm'] /
  ↳ 1e6) * df.loc[need_rp, 'star_radius_Rsun'] *
  ↳ 109.1
17 df.loc[need_rp, 'planet_radius_Re'] = Rp_est
```

```
20 # Imputación metallicity con mediana global
21 met_med = df['star_metallicity'].median()
22 df['star_metallicity'] =
  ↳ df['star_metallicity'].fillna(met_med)
23
24 # Verificación
25 print(df.isnull().sum())
```

Discusión: La imputación de Rp usa la relación física del tránsito; el resto usa medianas robustas por grupos.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 3:

Columnas derivadas físicas



Enunciado

- Recalcula `a_AU` con Kepler: $a \propto P^{2/3} M_{\star}^{1/3}$.
- Calcula `teq_K` con albedo 0.3: $T_{eq} \approx T_{eff} \sqrt{\frac{R_{\star}}{2a}} \cdot (1 - 0.3)^{1/4}$, usando $R_{\odot}/\text{AU} = 0.00465047$.
- Clasifica `planet_type`: *Terreno* ($< 2 R_{\oplus}$), *Sub-Neptuno* (2–4), *Gigante* (> 4).

Conceptos: Uso de unidades y umbrales astrofísicos comunes.

Ejercicios Prácticos Integrados $\ni$ Solución 3 de Referencia:

Columnas derivadas físicas



```
1 # a_AU (recalcular por consistencia)
2 df['a_AU'] = (df['period_days']/365.25)**(2/3) * (df['star_mass_Msun']**(1/3))
3
4 # Temperatura de equilibrio
5 Rsun_AU = 0.00465047
6 geom = (df['star_radius_Rsun'] * Rsun_AU) / (2.0 * df['a_AU'])
7 albedo = 0.30
8 df['teq_K'] = df['star_teff'] * np.sqrt(geom) * ((1 - albedo)**0.25)
9
10 # Clasificación por tamaño
11 def classify_rp(r):
12     if r < 2.0: return 'Terreno'
13     if r <= 4.0: return 'Sub-Neptuno'
14     return 'Gigante'
15 df['planet_type'] = df['planet_radius_Re'].apply(classify_rp)
```

Discusión: La T_{eq} es una cota simplificada; útil para filtros de “zona habitable aproximada”.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 4: Candidatos templados y detectables



Enunciado

- Selecciona candidatos con $250 \leq \text{teq_K} \leq 330$,
 $\text{planet_radius_Re} \leq 2.5$, $\text{period_days} \in [20, 400]$, $\text{snr} \geq 7$.
- Reporta conteo y muestra las 10 mejores SNR.

Objetivo: Filtro físico + criterio de detectabilidad.

Ejercicios Prácticos Integrados $\ni$ Solución 4 de Referencia: Candidatos templados y detectables



```
1 mask = (  
2     df['teq_K'].between(250, 330) &  
3     (df['planet_radius_Re'] <= 2.5) &  
4     df['period_days'].between(20, 400) &  
5     (df['snr'] >= 7)  
6 )  
7 candidatos = df[mask]  
8 print("Candidatos:", len(candidatos))  
9 candidatos.sort_values('snr', ascending=False).head(10)[  
10     ['id', 'planet_radius_Re', 'period_days', 'teq_K', 'snr']  
11 ]
```

Discusión: El filtro combina habitabilidad aproximada con SNR mínima típica de detección.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 5:

Tasas por metalicidad y tipo de planeta



Enunciado

- Discretiza `star_metallicity` en cuartiles.
- Calcula proporción de *Terreno/Sub-Neptuno/Gigante* por cuartil.
- Interpreta tendencia: ¿mayor metalicidad favorece gigantes?

Conceptos: `pd.cut`, `groupby`, proporciones.

Ejercicios Prácticos Integrados $\ni$ Solución 5 de Referencia: Tasas por metalicidad y tipo



```
1 met_bins = pd.qcut(df['star_metallicity'], 4, labels=['Q1','Q2','Q3','Q4'])
2 tab = (df
3       .groupby([met_bins,'planet_type'])
4       .size()
5       .groupby(level=0)
6       .apply(lambda s: s/s.sum())
7       .unstack(fill_value=0)
8 )
9 print(tab.round(3))
```

Discusión: Suele observarse mayor fracción de gigantes a metalicidades altas (tendencia plausible).



Enunciado

- Histograma de `period_days` en escala log.
- Dispersión `period_days` vs `planet_radius_Re` coloreado por `teq_K`.
- Barras: conteo por `planet_type`.

Objetivo: Explorar distribución, relaciones y composición de la muestra.

Ejercicios Prácticos Integrados $\ni$ Solución 6 de Referencia:

Gráficas



```
1 import matplotlib.pyplot as plt
2
3 # Histograma de periodos (log)
4 plt.figure()
5 plt.hist(df['period_days'], bins=40)
6 plt.xscale('log')
7 plt.xlabel('Periodo [d] (log)');
8 plt.ylabel('Frecuencia')
9 plt.title('Distribución de periodos');
10 plt.grid(alpha=0.3)
11 plt.show()
12
13 # Dispersión P vs Rp con color por Teq
14 plt.figure()
15 sc = plt.scatter(df['period_days'],
16                 df['planet_radius_Re'],
17                 c=df['teq_K'], cmap='plasma',
18                 s=10, alpha=0.7)
19 plt.xscale('log'); plt.xlabel('Periodo [d] (log)')
20 plt.ylabel('Radio planetario [RJ]')
21 plt.title('Periodo vs Radio (color = Teq)')
22 plt.colorbar(sc, label='Teq [K]')
23 plt.grid(alpha=0.3); plt.show()
```

```
23 # Barras por tipo de planeta
24 plt.figure()
25 df['planet_type'].value_counts().plot(kind='bar',
26                                       rot=0)
27 plt.ylabel('Conteo'); plt.title('Tipos de planeta')
28 plt.grid(axis='y', alpha=0.3)
29 plt.show()
```

Discusión: Patrones esperados: más periodos cortos, relación P-Rp difusa, mezcla de tipos por formación/migración.



Enunciado

- Crea `df_clean` con columnas clave y sin nulos restantes.
- Exporta a `exoplanetas_sintetico.csv`.
- Exporta top-50 `candidatos` del Ejercicio 4.

Objetivo: Dejar resultados persistentes para análisis externo.

Ejercicios Prácticos Integrados $\ni$ Solución 7 de Referencia: Exportación



```
1 cols = ['id', 'star_teff', 'star_radius_Rsun', 'star_metallicity', 'star_mass_Msun',  
2         'period_days', 'planet_radius_Re', 'a_AU', 'depth_ppm', 'snr', 'teq_K', 'planet_type']  
3 df_clean = df[cols].dropna()  
4 print(df_clean.shape)  
5  
6 df_clean.to_csv('exoplanetas_sintetico.csv', index=False)  
7 candidatos.sort_values('snr', ascending=False).head(50).to_csv(  
8     'exoplanetas_candidatos_top50.csv', index=False  
9 )
```

Discusión: Exportar vistas “limpias” facilita reproducibilidad y colaboración.

Cierre y Retroalimentación

- **Estructuras y conceptos integrados:**
 - pandas: inspección, imputación, derivaciones, agrupación, exportación.
 - Visualización: histogramas, dispersión, barras.
- **Buenas prácticas observadas:**
 - Imputaciones guiadas por física (tránsito).
 - Umbrales claros de filtrado físico y SNR.
 - Unidades consistentes y comentarios.

- ¿Qué estrategia de imputación funcionó mejor y por qué?
- ¿Qué patrones observaron en P vs R_p y en T_{eq} ?
- ¿Cómo cambia la composición por metalicidad?

Importante

La interpretación física guía el análisis de datos y el tratamiento de faltantes.

Conclusiones

- **Consolidamos:** flujo pandas completo con un caso astronómico realista.
- **Aplicamos:** relaciones físicas simples para enriquecer el análisis.
- **Desarrollamos:** criterio para filtros y evaluación de candidatos.

- **Próximamente:** POO avanzada y análisis más profundo con pandas.
- **Recomendación:**
 - Practicar imputación por modelos y validación cruzada.
 - Explorar visualizaciones interactivas.

¡Nos vemos en la próxima sesión!